

Herald



Herald — Fixed

Field	Value
Revision	17
Created	2026-05-20
Last modified	2026-05-29
Status	active
Status summary	r17 atomically lands (§11.4.19) the completed Wave 8 Track B (MTPROTO user-account harness for §11.4.98 full-automation testing) — HRD-139 (MTPROTO Client live wiring + qaherald mtpROTO CLI), HRD-140 (Subscribe roundtrip), HRD-141 (Wave 6 closed loop), HRD-142 (Wave 6.5 lifecycle) all New→Fixed (these HRDs were tracked directly in the TaskList rather than Issues.md and graduate directly into Fixed.md per §11.4.19 the moment closure landed). Wave 8 Track B closure proves §11.4.98 (full-automation anti-bluff mandate) end-to-end: 3 MTPROTO-driven live tests (E135 Subscribe / E136 Wave 6 closed loop / E137 Wave 6.5 lifecycle) all PASSING LIVE under §11.4.98-pure interpretation, captured under docs/qa/HRD-LIVE-MTPROTO-20260528T125321Z/. Three legacy NON-COMPLIANT tests graduate to §11.4.90 Obsolete in Issues.md r36 with Obsolete-Details cite of §11.4.98 + §108.m superseding-mandate: TestSubscribe_LiveBotAPI, tests/test_wave6_live_loop.sh, lifecycle --manual mode. v0.6.0 tag (844f340 annotated) was already cut on d843ddf covering Wave 8 Track B closure + the §11.4.98+§11.4.99 anchoring + 9-Helix-submodule §11.4.98+§11.4.99 propagation. Prior r16 atomically migrates (§11.4.19) the completed Tgram-completeness wave — HRD-133/134/135/136/137/138 (all 6 Tgram-completeness HRDs) Issues→Fixed.** Wave A: HRD-135 lifecycle handlers (fe2a7c8 — OnEdited / OnMyChatMember / serviceHandler / ErrBotKicked sentinel) + HRD-136 webhook secret_token validator (6d7d543 — crypto/subtle.ConstantTimeCompare, 401-no-leak posture) + HRD-138 operator deep-dive guide (ff9b9c0 — 810-line docs/guides/TELEGRAM.md, 11 §-sections, 3 siblings). Wave B: HRD-133 token-redaction symmetry (8162d14 — sanitizeTgramError wraps 6 error-return sites in Send/SendReply) + HRD-134 429 retry_after honored (8162d14 combined — withRetry429 wraps all 3 telebot bot . Send sites, telebot.FloodError typed extraction, ctx-aware sleep, 3-attempt 60s-clamp). Wave C: HRD-137 stress+chaos suite (this commit — TestTgram_Stress_MultiRecipientFanOut 20×20=400 calls/20 chats deterministic + TestTgram_Chaos_GetUpdatesPollerResilience SKIP→PASS

Field	Value
	conversion via one-line <code>subscribe.go</code> baseURL seam fix; 8-phase fault matrix deterministic across <code>-race -count=3</code> — every iteration IDENTICAL recovery=6 fault(500=12 hang=8 close=3 ok=6) goroutines 3→4). Full tgram suite 26/26 PASS, 0 SKIP post-Wave-C. All 6 HRDs carry docs/qa/HRD-NNN-20260528T*Z/transcript.txt evidence per §107.x. Prior r15 atomically migrated (§11.4.19) the completed GAP-3 §11.4.85 stress+chaos suite — HRD-122..HRD-130 (Task→Completed) — and the HRD-132 Runner idempotency-window fix (Bug→Fixed): 10 rows Issues→Fixed.** GAP-3 commits 02d272d→d2a60d3 (HRD-122 commons/stresschaos/scaffold, HRD-123 Gin /v1/events, HRD-124 /v1/compliance+/v1/safety_state, HRD-125 Runner — surfaced HRD-132, HRD-126 pherald listen inbound, HRD-127 claude_code dispatch, HRD-128 container/resource §12.6-safe, HRD-129 e2e E81-E88, HRD-130 paired §1.1 mutation gate); HRD-132 fix 354b883 (claim-before-dispatch at Runner Stage 2 — INSERT events_processed ... ON CONFLICT DO NOTHING is the authoritative exactly-once dispatch gate; latent runner.go:132 CachedRcpt.WasReplay race closed; HRD-125 assertion upgraded to sends==1) + wave3-M4-retire 4f45f26. Full scripts/e2e_bluff_hunt.sh 78 PASS / 0 FAIL; HRD-130 gate 6 PASS / 0 FAIL (5 load-bearing mutations); HRD-132 evidence docs/qa/HRD-125-stress-chaos-2026-05-27T131251-gap3/(dispatch_exactly_once=1). HRD-131 (SQLite SSoT migration Phase 1) remains OPEN. Prior r14 captured Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure). HRD-100 closed atomically with 12 Wave-6 commit SHAs + 8 new e2e invariants E63-E70 (currently SKIP-with-documented-reason — converting to PASS once operator runs the live closed-loop at T10b and commits docs/qa/HRD-100/). Tag v0.4.0 DEFERRED to T13b (operator-supplied live evidence per §107.x — code-doc closure ships in this commit; runtime evidence lands separately).** Wave 6 surfaces: new Cobra subcommand pherald listen (Telegram getUpdates long-poll + OnText/OnPhoto/OnDocument/OnVoice + bot self-filter + sha256 content-addressed attachment download); Opus model pin in spawned claude argv(--model claude-opus-4-7); envelope pre-text — verbatim operator wording "We have received new message from our communication channel. ..." preceding the <<<HERALD-DISPATCH-v1>>> structured block; <<<HERALD-REPLY>>> parser with action routing (reply/issue.open/event.emit); tgram.Adapter.SendReply wiring reply_to_message_id; --qa-out-dir JSONL journaling per §107.x. Wave 6 mutation gate (tests/test_wave6_mutation_meta.sh) lands 3 paired hermetic mutations (M1 blank bot self-filter → echo-loop signature; M2 Opus → Sonnet model swap → E67 FAIL; M3 drop opts.ReplyTo in SendReply → E68 FAIL). Prior r13 captured Wave 4b (TOON application/toon content negotiation; tag v0.3.0). r12 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3; tag v0.2.0). r11 closed HRD-016 (Wave 3b pherald Runner live). r10 closed HRD-011 (Telegram live). see Issues.md — r35 (Issues.md) captures the HRD-133/134/135/136/137/138
Issues	Issues→Fixed atomic migration recorded in this r16; prior r16 (Issues.md) captured the HRD-122..HRD-130 + HRD-132 Issues→Fixed atomic migration recorded in the prior r15.
Issues summary	HRD-008/-015 + HRD-019..HRD-025 + HRD-029..HRD-056 + HRD-081 + HRD-085..HRD-090 + HRD-115..HRD-121 + HRD-131 still open (31 items).
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-018, HRD-026, HRD-027,

Field	Value
	HRD-028, HRD-080, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099, HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114, HRD-122, HRD-123, HRD-124, HRD-125, HRD-126, HRD-127, HRD-128, HRD-129, HRD-130, HRD-132, HRD-133, HRD-134, HRD-135, HRD-136, HRD-137, HRD-138, HRD-139, HRD-140, HRD-141, HRD-142
Fixed summary	Wave 8 Track B (HRD-139/140/141/142 — MTProto user-account harness proves §11.4.98 full-automation end-to-end with 3 live MTProto-driven tests PASSING LIVE 2026-05-28; v0.6.0 tag covers the closure); Tgram-completeness wave (HRD-133/134/135/136/137/138 — 6 HRDs across 3 sub-waves A/B/C, full tgram suite 26/26 PASS 0 SKIP); GAP-3 §11.4.85 stress+chaos suite (HRD-122..HRD-130 Completed) + HRD-132 Runner idempotency-window fix (claim-before-dispatch, exactly-once dispatch <code>sends==1</code>); spec V1→V3 r9; Go module foundation + Foundation M1/M2/M3; HRD-010 commons_storage live wiring; HRD-011 Telegram live (live message_id=5 evidence 2026-05-22); HRD-012 Claude Code dispatcher live; HRD-016 pherald /v1/events Runner wiring (Wave 3b live — 7-stage pipeline + e2e E37-E42); universal §11.4.73 + §11.4.74 mandates propagated; I6 gate refined; Wave 2 flavor scaffolds (HRD-092..097) closed atomically; Wave 3a substrate (HRD-099 commons_auth JWT verifier) + 2 live REST routes (HRD-028, HRD-098) closed atomically.

Continuation see CONTINUATION.md.

Table of contents

- [Recently fixed](#)

Recently fixed

ID	Type	Criticality	Title
HRD-133	task	high	Tgram-completeness Wave B — token-redaction symmetry (<code>commons_messaging/channels/tgram/errors.go</code> + <code>send.go</code> wraps). <code>sanitizeTgramError(token) string</code> strips the bot token from every <code>Send/SendReply</code> error message so it can NEVER appear in observable output — 6 error-return sites wrapped (<code>errors.ParseInt/withRetry429</code> paths in both <code>Send</code> + <code>SendReply</code>). <code>TestTgram_Send_ErrorDoesNotLeakToken</code> plants <code>123456:supersecret</code> must-not-leak-9999, drives an <code>httptest</code> 401 path, asserts the token substring from the returned <code>err.Error()</code> (mirrors slack's <code>TestSlack_Send_ErrorDoesNotLeakToken</code>). Unit-level <code>TestTgram_SanitizeTgramError_Unit</code> (4 subtests) pins the helper. Traces captured in the source: <code>%w</code> error-wrap replaced with <code>%s</code> + <code>sanitizeTgramError(err.Error(), token)</code> since <code>strings.Repl</code> operates on the string form — the error chain is sacrificed to guarantee the secret Composes with HRD-134 (the 429 retry's log line + final error are themselves sanitized 429 logging never leaks the token).
HRD-134	task	high	Tgram-completeness Wave B — Bot API rate-limit 429 + <code>retry_after</code> honored (<code>commons_messaging/channels/tgram/send.go</code> + <code>tgram.go</code> sleeps <code>errors.go::extractRetryAfter</code>). <code>withRetry429(ctx, fn) error</code> 3 telebot bot. <code>Send</code> call sites (<code>Send</code> text body + <code>SendReply</code> text + <code>SendReply</code> per fan-out). On <code>429 + retry_after=N</code> → sleep N seconds (<code>ctx.Done()</code> -aware) up to 3 attempts total, clamp per-retry wait to 60s (defense-in-depth against Bot API

ID

Type Criticality Title

HRD-135 task middle

violations). `retry_after` extracted via `errors.As(err, &telebot.FloodError{})`. `RetryAfter` (telebot's typed error — NOT b
Logged retry line goes through `sanitizeTgramError` so the token stays out
(composes with HRD-133). Injectable `sleeper` field on `Adapter` for hermetic
time + fake-clock both supported via `SetSleeperForTest`). 4 hermetic `http`
(`HonorsRetryAfter` / `GivesUpAfter3Attempts` / `RespectsCtxCancel` / `ClampsRetry`
+ 1 unit test (`ExtractRetryAfter_Unit`) — 5 total, all `-race -count=1` green.
Tgram-completeness Wave A — edited-message + bot-kicked typed-error + servi
ignore-and-log handlers (`commons_messaging/channels/tgram/subs`
`bot.Handle(telebot.OnEdited, ...)` builds the same `InboundEvent`
`OnText` does + `Raw["edited"]=true`;
`bot.Handle(telebot.OnMyChatMember, ...)` → `botKickedFromUp`
detects status transitions to `kicked/left/restricted` **for this bot specifically** (Alice-l
promotion-to-Admin false-positive guarded) and emits a grep-able `log.Print`
bot kicked from chat: `chat_id=... new_status=... ErrBotKic`
serviceHandler registered against 15 telebot service `On...`-endpoints (`UsersJ`
`UserLeft` / `NewGroupTitle` / etc.) emits NOTHING to `InboundHandler` + logs `tgr`
ignored service message `kind=... chat_id=...`. Sentinel var `ErrE`
= `errors.New("tgram: bot kicked from chat")` package-level (c
`errors.Is`). The `OnMyChatMember` closure returns `nil` deliberately so telebot
keeps running for OTHER chats. 17/17 PASS `-race -count=1` (3 new in
`subscribe_lifecycle_test.go` + 14 existing, zero regression).

HRD-136 task low

Tgram-completeness Wave A — Bot API webhook `secret_token` validation s
(`commons_messaging/channels/tgram/webhook.go`). `WebhookHan`
`http.Handler` validates X-Telegram-Bot-API-Secret-Token via crypt
`subtle.ConstantTimeCompare` (timing-attack-safe); refuses empty secret
construction time (`NewWebhookHandler("", ...)` errors loud — §107 footgun
returns 401 with empty body on header mismatch / missing header (no token-exis
leakage to probes); 400 on malformed JSON body; 405 on wrong method (POST
(no body) on `InboundHandler` error. Forwards a decoded `tele.Update` as the S
`InboundEvent` shape `OnText/OnEdited` produce — unified downstream co
Skeleton-only: wiring into pherald `listen --webhook` + the
`setWebhook(secret_token=...)` registration + the public-endpoint TLS lis
deferred to V1.x; the cryptographic-validation layer is here + tested so the future
flag affair. 7/7 hermetic `http`test tests `-race -count=1` PASS: `ValidSecret_Di`
`InvalidSecret_Returns401_NoTokenLeak`, `MissingSecret_Returns401`, `GET_Retu`
`MalformedJSON_Returns400`, `EmptySecretAtConstruction_ErrorsLoud`,
`ConstantTimeCompare`.

HRD-137 task high

Tgram-completeness Wave C — §11.4.85 stress + chaos suite (`commons_mess`
`channels/tgram/stress_chaos_test.go`).
`TestTgram_Stress_MultiRecipientFanOut` drives the REAL
`Adapter`. `SendReply` via `NewAdapterWithBaseURL` `http`test seam under
× 20 iterations = 400 total calls against 20 UNIQUE `chat_ids` on the SAME
*`tgram.Adapter`; asserts no-loss (every `chat_id` received exactly 20 hits), no-
(global hit counter == 400), no-confusion (uniform per-chat distribution, no cross
`-race-clean`, throughput ≥ 100 ev/s (observed 5215-6422 ev/s, p99=9.73-15.20ms)
`TestTgram_Chaos_GetUpdatesPollerResilience` converted from t
SKIP-with-reason to a real `http`test-driven fault-injector PASS via a ONE-LINE
`subscribe.go` seam fix: `threads a.baseURL` into `telebot.Settings.L`
`NewBot` (parallel to the `ensureBot` pattern at `tgram.go:60-67`; production
unchanged when `a.baseURL == ""`). 8-phase fault matrix (3× 500 + 2× 1.5s
honoring `ctx-cancel` + 1× mid-body Hijacker connection-close + 2× success), bot-

ID

Type Criticality Title

HRD-138 task low

aware self-filter bypass (success messages come from @someuser, not chaotestbot). Deterministic across -race -count=3 per §11.4.50 — all IDENTICAL: recovery_event_count=6 fault_modes(500=12 handle_close=3 ok=6) subscribe_err=context deadline exceeded goroutines 3→4 (leaked=1) duration=12.07s. Full tgram suite 26 **SKIP** post-fix (was 24/24 with chaos SKIPped). §11.4.92 5-pass: P1 captured-event iteration table; P2 blast-radius — subscribe.go baseURL seam is read-only v (production unaffected); P3 cross-feature — the seam mirrors existing ensureB no novel state; P4 deep-research — telebot.v3 LongPoller retry behavior under (poller logs + continues, no internal retry); P5 anti-bluff — handlerHits, fault-count ctx-aware subscribe return, no goroutine leak (delta=1) all positive captured invariant REAL Subscribe poller, not synthetic.

Tgram-completeness Wave A — docs/guides/TELEGRAM.md per-module or deep-dive guide (810 lines, 11 §-sections, 58 subsections, 3 siblings .html/.pdf/.doc architecture overview (inbound getUpdates + outbound Send/SendReplyG §2 BotFather walkthrough (/newbot → token → /setprivacy); §3 required how to obtain HERALD_TGRAM_CHAT_ID (3 methods: getUpdates curl / @user @RawDataBot); §4 pre-deploy smoke-test commands; §5 attachment size limits (10/50/2GB photo tiers, 50MB document Bot-API-cap workaround = local Bot API the content-addressed inbox path; §6 chat-type matrix (private/group/supergroup/ formats + required bot permissions per type); §7 reply threading via reply_to_message_id + the attachment fan-out rationale (Wave 6.5 mutation §8 self-filter contract (cross-the-wire getMe + cache + the echo-loop hazard if sk Wave 6 T12 M1); §9 12-entry troubleshooting cookbook (401/400/429/413/403/e long-poll-stuck/409/privacy/webhook/zero-byte/wrong-inbox); §10 6-subsection operator audit (tick-box checklist mirroring OPERATOR_CREDENTIALS.md style references (spec V3 §11.1/§32.2/§32.9/§43 + 13 HRDs + source paths). Cross-link existing docs/guides/messengers/TELEGRAM.md (HRD-011 setup, pre- untouched).

HRD-139 task high

Wave 8 Track B — qaherald/internal/mtproto/ MTProto user-account wiring (gotd/td v0.144.0 vendored as submodules/gotd-td per §11.4.74 cat Client interface (Connect / SendMessage / WaitForReply / WhoAmI / CL backed by gotd/td's telegram.Client with session-storage at ~/.config/ mtproto.session (mode 0600, encrypted MTProto session blob). qaherald mtproto login/whoami/logout Cobra subcommands drive the AuthFlow app_hash + phone-from-env, 2FA password from env, OOB-code via stdin (TTY HERALD_MTPROTO_ALLOW_NON_TTY=1 env override enables delegated-by-c FIFO-piped flow for §11.4.98-compliant headless login). Conductor-driven end-to operator's single contribution = one-time paste of 5-digit Telegram OOB code at f login (Telegram fundamentally requires this; session persists thereafter for unlimi HRD-133 sanitizer verified leak-free across all evidence (token-shape regex + 32 api_hash regex; pherald.pid runtime artifact removed; .gitignore hardened).

HRD-140 task high

Wave 8 Track B Test 1 — TestMTProto_Subscribe_AutonomousRound (qaherald/internal/lifecycle/mtproto_subscribe_test.go, integration_mtproto). §11.4.98 replacement for the legacy TestSubscribe_LiveBotAPI Bot-API-only path which is fundamentally not autonomous (a bot can never send-to-itself, so the legacy test required an operator chat). New test drives the closed loop end-to-end via MTProto user-account: Client.SendMessage(chat_id, stimulus) → poll bot.getUpdates for delivery confirm assert message delivered to bot within timeout. Honest-SKIP per §11.4.3 when M absent OR session-file missing (never falls back to manual-dep path). Conductor-

ID

Type Criticality Title

HRD-141 task high

PASSING LIVE in 1.30s under §11.4.98-pure interpretation (chain runs without h during execution).

Wave 8 Track B Test 2 — TestMTPProto_Wave6_AutonomousClosedLoc (qaherald/internal/lifecycle/mtproto_wave6_loop_test.go integration_mtproto). §11.4.98 replacement for tests/ test_wave6_live_loop.sh (which required manual operator typing). Spaw pherald listen as a child process with cmd.Dir = repoRoot (CRITIC pherald resolves docs/Issues.md relative to its CWD), pre-acks pending upd consumePendingUpdates(t, botToken) helper (calls getUpdates(offset=max+1) to ACK stale chat messages BEFORE pherald preventing TLS-connection-reset storms from prior runs). MTPProto sends stimulus journal-based §11.4.98 autonomy assertion polls pherald's --qa-out-dir JSO transcript for tgram.message (in) + cc.dispatch + cc.reply directions the unique test-text string rather than message_id (MTPProto's int64 message_id is namespace from Bot API's chat-local message_id — §11.4.98 contract is "chain r end without human action", not "Claude generates perfect content"). PASSED LI 48.58s.

HRD-142 task high

Wave 8 Track B Test 3 — TestMTPProto_Wave65_LifecycleAutonomou (qaherald/internal/lifecycle/mtproto_wave65_lifecycle_t build-tag integration_mtproto). §11.4.98 replacement for the legacy lifec manual interactive mode (which required operator typing per-scenario stimuli). representative Wave 6.5 fast-path scenarios — Help / Status / Continue — autonom MTPProto; each subtest spawns pherald + sends MTPProto stimulus + asserts on the journal for tgram.message in + `cc.reply

HRD-018 task high

commons_constitution Go package: Evaluator interface + 12 event-class e + constitution_state/constitution_bindings migrations + bundle captureer + mode-ladder runtime config + **constitution_audit write-through (th gap)**. The Evaluator framework, emit helpers, bundle-hash, mode-ladder, state state.Postgres, ladder.Memory/ladder.Postgres, and migrations 000006/000007/000008 were all coded in r1. v1.0.0 Batch A closed the remaining PASS-bluff: migration 000006 defined constitution_audit but NO code e row — Runner.Run set RunOutcome.Audited=true with nothing persist an AuditStore interface (audit.go) + state.MemoryAudit (append-on state.PostgresAudit (RLS-guarded INSERT-only into constitution_ append-only is enforced by the migration's no-update/no-delete RLS policies). Ru now writes a durable audit row for every CHANGED transition in ModeWarn/Mode recording old/new decision, old/new digest, bundle hash, mode-at-emission, and - ModeEnforce — the EXACT emitted event ID (new IDEmitter optional interf PolicyViolationID/PolicyClearedID; emit returns the generated even NewRunner gained a required AuditStore arg (nil = hard error). §11.4.92 multi-pa TDD RED (compile-fail before backends) → GREEN, -race -count=3 dete Pass-5 real DB-level SELECT shows enforce rows carry emitted_event_id, warn

HRD-026 task middle

Constitution-bundle hash captureer — computes SHA-256 of rendered Constitution evaluation time; persists on every emitted event for replayability. The captureer (l Capture/CaptureBytes/Captureer) was coded in r1 and the hash already emitted-event metadata (bundle_hash hex). v1.0.0 Batch A completed the pers the 32-byte bundle_hash is now stored on every constitution_state row every constitution_audit row (verified by the DB-level SELECT ... encode(bundle_hash, 'hex') in the evidence dir showing a 64-hex SHA- every audit row), closing the §42.1.3 replay-correlation requirement.

HRD-027 task middle

ID

Type Criticality Title

HRD-132 bug high

Mode-ladder runtime config (constitution_bindings table + admin REST to flip allow/warn/enforce per binding per tenant without redeploy). The Model interface + ladder.Memory/ladder.Postgres (reading constitution_bindings at evaluation time) were coded in r1.v1.0.0 Batch the admin REST surface: cherald/internal/modes — GET /v1/compliance/modes (list bindings + unbound-default), GET /v1/compliance/modes/: (one binding's effective mode, enforce default), PUT /v1/compliance/modes/:rule_id (flip allow/warn/enforce; records the operator sub claim in mutated_by). JWT-gated via the existing commons_auth.GinMiddleware mounted in cherald serve (a single shared PG pool backs both compliance + m anti-bluff: 401 no-auth / 401 bad-tenant / 400 bad-mode (loud, never a silent default bearing proof: a PUT flip to warn is reflected by a subsequent GET AND by ladder on the evaluation hot path — runtime config, no redeploy).

§11.4.85/§32.1 Runner idempotency-window FIX — concurrent same-key replay duplicate-dispatches a notification (FINDING from HRD-125 stress/chaos). Root idempotency.go:64-72 treated a Redis-SETNX-miss-with-no-PG-row as F §32.1 "Redis-lies-PG-truths" design) while the events_processed archive row written at Stage 7 (OutcomeRecorder.Process) — so concurrent same-key arriving in the Stage-2→Stage-7 window ALL dispatched (observed 2–4 of a 100 archival exactly-once via PG UNIQUE + ON CONFLICT DO NOTHING, but cl dispatch only bounded = duplicate-alert delivery under load). Fix: claim the idempotency atomically BEFORE dispatch — INSERT events_processed ... ON CONFLICT DO NOTHING moved to Stage 2 (the PG row, serialized by PRIMARY KEY(tenant_id, idempotency_key), is now the AUTHORITATIVE exactly-once dispatch gate; 0-r affected = duplicate → no dispatch), preserving the nil-Redis PG-only degrade. A the latent runner.go:132 CachedRcpt.WasReplay data race (mutated a shared *Receipt). The §11.4.85 HRD-125 stress test was upgraded to assert se under the same 1000× shared-key flood — the stronger assertion HRD-125 deliberate NOT make pre-fix (§11.4.4 test-interrupt-on-discovery). §11.4.92 multi-pass: Pass independent -race -count=3 runner + http + storage green; Pass-5 sends=true (RED proved before fix). wave3 mutation-gate M4 retired (4f45f26) — Stage now redundant, M4's blast-radius moved into the Stage-2 claim.

HRD-130 task high

§11.4.85 / §1.1 paired mutation gate for the GAP-3 stress+chaos suite — tests.test_stress_chaos_mutation_meta.sh (GAP-3 plan §5 / §6 T9). Prov hermetic stress/chaos detector is genuinely LOAD-BEARING via the canonical test_wave6.5_mutation_meta.sh idiom: .git/MUTATION_IN_PROGRESS.lockfile, dirty-tree + pre-existing-residue refusal pre-flights, run_paired (apply mutation → assert marker landed → run detector EXPECT-FAIL → git checkout → byte-for-byte assert_restored vs HEAD → re-run EXPECT-PASS), and check_quiescence scan asserting no MUTATED SC-M* markers leaked (§1 paired mutations, each against a hermetic detector: SC-M1 drop ctx from claude_dispatch.go exec.CommandContext→exec.Command → timeout-cancel no longer cancels the 30s sleep → watchdog FAIL; SC-M2 swallow the claude_code parseReply decode error → truncated-reply test returns nil → FAIL; SC-M3 inbound extractReplyToMessageID default: panic instead of return → degrade test panics → FAIL; SC-M4 mismatch events mapErrorToStatus event_parser: 400→500 → input-corruption test sees 5xx → FAIL; SC-M5 commons_storage applyMigration disk-full write error → ENOSPC reports silent-swallow guard FAIL. Runs ONE-AT-A-TIME, FOREGROUND (§107.y + 2026-05-27 concurrent-gate-residue memo). Conductor-executed result: 6 PASS / mutations load-bearing).

HRD-129 task high

ID

Type Criticality Title

HRD-128 task high

HRD-127 task high

§11.4.85 stress+chaos suite wired into the canonical anti-bluff harness `scripts/e2e_bluff_hunt.sh` as new invariants E81-E88 (GAP-3 plan §4 / §6 T8) — ADDITIVE-ONLY append after E80 (E1-E80 logic untouched). Each invariant REAL hermetic Go stress/chaos test via the existing check helper AND cites the evidence artefact under `docs/qa/<run-id>/stress_chaos/` as its §11.4. positive-evidence anchor through a new `sc_anchor` helper that greps a SPECIFIC bearing value (present-but-empty artefact FAILs as a §11.4 PASS-bluff; absent ev SKIPs-with-reason). Mapping: E81 runner exactly-once-archive + bounded dispatch (HRD-125); E82 `/v1/events` input-corruption + duplicate-key + auth-storm (HRD-126); E83 `/v1/compliance` fail-loud (HRD-127); E84 `/v1/safety_state` concurrent-mutation (HRD-124); E85 pherald 1 inbound malformed-Raw panic-free + extract-degrade (HRD-126); E86 `claude_code` death + timeout-cancel + truncated-reply (HRD-127); E87 disk-full tagged-error (HRD-128); E88 §12.6 host-mem headroom (HRD-128). All eight hermetic; live pause / real-disk-fill / container-OOM variants are SKIP-with-reason inside the G (HERALD_STRESS_LIVE_*=1 gated). Verified `bash -n` clean; E81-E88 block PASS deterministic; header bumped E1..E80→E1..E88. Full `scripts/e2e_bluff_hunt.sh` reports 78 PASS / 0 FAIL (was 60; +18).

§11.4.85 container-orchestrated / resource-exhaustion stress+chaos tests (GAP-3 plan §6 / §6 T7 + §7 host-safety) — `commons_storage/resource_stress_chaos_test.go` (hermetic core) + `tests/test_resource_stress_chaos.sh` (LIVE container harness). HERMETIC deterministic): (1) disk-full error propagation — a fake database. Database. `syscall.ENOSPC` on the migration-UP exec, driving the REAL RunMigration write path; asserts real write path exercised, error NOT swallowed. tagged `commons_storage: apply + exec up, errors.Is(err, syscall.ENOSPC)` true, 0 migrations reported applied. (2) §12.6 host-mem headroom `HostMemHeadroom()` pre/post a 25× workload shows `used_fraction_de` (host needle did not move) = §12.6 compliance evidence. OPT-IN (deterministic without env): real bounded-scratch disk fill HARD-capped 64 MiB, gated `HERALD_STRESS_LIVE_DISK=1`. LIVE container scenarios SKIP-with-reason (host-safety-gated): container OOM-confinement + connection-churn against a memory-bounded PG, gated `HERALD_STRESS_LIVE_OOM=1` + a §12.6 pre-flight headroom gate that REFUSES to add pressure when host `used-fraction` ≥ 60% (host ~74% this run → honest SKIP). §12/§12.6 honoured: NO host-OOM, NO real-host NO host process killed. `go test -race -count=3` green (PASS×3/PASS×3); `go vet` clean.

§11.4.85 `claude_code` dispatch stress+chaos tests (GAP-3 plan §1 row 5 / §6 T6) — `commons_messaging/dispatch/claude_code/dispatch_stress_chaos_test.go` (in-package, exercises unexported `bootstrapSession/parseReply`) + 5 committed hermetic shims `testdata/claude-*.sh`. Per §11.4.27 only the EXTERNAL `claude` CLI is faked via the `New(binaryPath,...)` seam; NO production source edited, NO new dep, NO new invoked. STRESS: (1) N=24×25=600 concurrent `Dispatch` (—`resume` path) every reply really parsed, every call resumed the SAME pre-seeded UUID, anchored, drifted, p50=62.9ms/p95=96.4ms; (2) N=16 concurrent COLD-START bootstrap succeed, exactly one anchor UUID persists, —`race` clean. CHAOS: (a) process-d 137 + `self-kill -KILL $$` → tagged error, no hang (watchdog), partial stdout accepted; (b) timeout — 30s-sleep shim vs 800ms ctx → cancellation fires, Dispatch ~803ms not 30s; (c) truncated <<<HERALD-REPLY>>> → explicit decode r JSON error, never silent partial-accept. FINDING-1: cold-start concurrent bootstrap

ID

Type Criticality Title

HRD-126 task high

HRD-125 task high

HRD-124 task high

HRD-123 task high

writer-wins by design (honest bound asserted, not `bootstrap_count==1` blue). FINDING-2: timeout-cancellation latency depends on `claude` being a single process (hardening direction = process-group kill, flagged for dispatcher owner). `go test -count=3` deterministically green $\times 3$.

§11.4.85 pherald `listen` inbound-path stress+chaos tests (GAP-3 plan §1 row 3) — `pherald/internal/inbound/stress_chaos_test.go` (in-package `unexported extractReplyToMessageID`) + `pherald/cmd/pherald/listen_stress_chaos_test.go` (drives the REAL `runListen` loop). Production only the channel `Subscriber` + `CC CodeDispatcher` boundaries are faked; the action-routing switch + `runListen` fan-in + clean-shutdown run unmodified. Stress: $N=16 \times 50=800$ concurrent `Dispatcher.Handle` — 0 errors, exactly-once reply (800/800), 0 goroutine leak, $p50=0.41\text{ms} \sim 25\text{k}$ dispatch/s; (2) 400-event burst through `runListen` — nothing dropped, clean shutdown on cancel ($<3\text{s}$), no leak. CHAOS: corrupt Raw payloads (nil map, missing/nil/string/bool/slice/map `message_id`) $\rightarrow$ `extractReplyToMessageID` degrades to (0,err); valid `int/int64/int32/float64` decode; 64-dispatch corrupt-Raw flood panic-free (recover-guard), degraded reply (b) CC subprocess fault (killed/truncated/marker-less/empty/unknown-action) $\rightarrow$ `inbound: error`, no reply leak, no hang; (c) once-only side-effect — 1000 redelivered same issue. open collapse to exactly-ONE durable sink row. `go test -race -count=3` deterministically green $\times 3$ on both packages. NO production source edited, new dep.

§11.4.85 Runner 7-stage pipeline stress+chaos tests (GAP-3 plan §1 row 3 / §6 T4) — `pherald/internal/runner/stress_chaos_test.go` exercising the `runner.Run` (only the external boundary — PG `events_processed` store, channel Redis SETNX seam — faked per §11.4.27; orchestrator + 7 stages run unmodified). Stress: $N=16$ goroutines $\times 40$ iters concurrent same-key replay + fresh keys, -race clean (a) 1000 $\times$ duplicate flood @ 50 parallel under live atomic Redis SETNX; (a') same nil-Redis PG-fallback degrade; (b) hermetic PG-deadlock (SQLSTATE 40P01) in the `events_processed.Insert`. SURFACED HRD-132: dispatch exactly-once was NOT guaranteed under concurrent replay (`events_processed` archived at Stage 7; observed duplicate dispatches of a 1000 $\times$ flood); tests asserted the honest code-true contract (exactly-once + bounded dispatch) pre-fix, then upgraded to `sends==1` once HRD-132 landed the Stage-2 claim. `go test -race -count=1` green.

§11.4.85 Gin /v1/compliance (cherald) + /v1/safety_state (sherald) stress+chaos tests (GAP-3 plan §1 row 2 / §6 T3) — `cherald/internal/compliance/compliance_stress_chaos_test.go` + `sherald/internal/safety/safety_stress_chaos_test.go`, each driving the REAL flavor over a REAL in-process `httpTest` server dialed by `net/http.Client` through production Gin chain (`cli.TOONMiddleware` $\rightarrow$ `auth` $\rightarrow$ `handler`). Only the external boundary faked per §11.4.27; NO production source edited, NO new dep. STRESS (flavor): $N=10 \times 200=2000$ GETs, -race clean — 0 errors, 0 5xx, all 200 (compliance req/s, `safety_state` $\sim 16\text{k}$ req/s). Content-negotiation correctness under concurrency: request alternates `Accept: application/toon` vs `application/json`; Content-Type match to its OWN `Accept` header (0 codec-bleed), every toon body wire bytes. CHAOS: (a) compliance PG-drop (`errStore`) $\rightarrow$ 2000/2000 fail-loads naming the failed dependency, 0 fabricated 2xx (a swallowed-error 200 {result} would falsely tell an operator "no violations" — the §107 mode this gate forbids); (b) has no hot-path DB by design so the code-true analogue (concurrent state-mutation load, -race-clean `RWMutex`) is exercised instead; (b) auth storm — 1000 random through the REAL `commons_auth.GinMiddleware+HMAC(HS256)` $\rightarrow$ 100% bypass. Verified `go test -race -count=3` green $\times 3$.

ID

Type Criticality Title

HRD-122 task middle

§11.4.85 Gin /v1/events stress+chaos tests (GAP-3 plan §1 row 1 / §6 T2) — internal/http/events_stress_chaos_test.go driving the REAL events handler (EventsHandler → runner.Run, all 7 stages) over a REAL process httptest server dialed by net/http.Client through the production (cli.TOONMiddleware + auth gate). Only the external boundary (PG/Redis/c runner.NewFakeRunner) faked per §11.4.27. STRESS: N=12×200=2400 C POSTs, -race clean — 0 errors, 0 5xx, all 202; p50=0.706ms ~13,322 req/s. CH duplicate-key-under-load (1200 POSTs, per-worker keys) → graceful degrade, 0 (c) input-corruption (8 MiB random, truncated JSON, non-UTF8, JSON array, mi garbage, empty) × 5 → every body a tagged event_parser: 4xx, 0 panic, 0 5 storm — 1000 random tokens through the REAL commons_auth.GinMiddleware+HMAC(HS256) → 1000×401, 0 bypass; drop → SKIP-with-reason (proven hermetically at HRD-125). FINDING (→ HR HTTP layer): a SHARED-key cross-worker concurrent replay flood triggers the F runner.go:132 CachedRcpt.WasReplay data race (fixed by HRD-132). re-captured as all_malformed_rejected_no_5xx (the §11.4.50 transport of-8-MiB-body is valid).

§11.4.85 stress+chaos shared scaffold — Go-native commons/stresschaos package (GAP-3 plan §2 / §6 T1). Provides: a bounded concurrent load-runner (R N workers × M iterations of a closure via stdlib sync.WaitGroup, no new dep nearest-rank percentile math (Percentiles → {p50, p95, p99, max, min, emitted as latency.json + latency_histogram.csv; an evidence-dir v creating docs/qa/<run-id>/stress_chaos/<surface>/; and a best-c memory headroom probe (HostMemHeadroom — vm_stat+sysctl hw.m darwin, /proc/meminfo on linux) for the §12.6 60%-ceiling proof (degrades t unavailable, never a hard failure). Self-tested: 8 unit tests (percentile correct known 1..100ms dataset, evidence-dir creation, bounded-goroutine proof, host-m effort, §12.6 ceiling predicate) PASS under go test -race -count=1. Lov layer per CLAUDE.md layering (commons L0, importable by every flavor).

HRD-114 task high

Wave 7 T5 — multi-channel pherald listen (HERALD_CHANNELS fan-i Subscribe goroutines → 1 dispatcher). pherald/internal/inbound/ dispatcher.go renames the Wave-6 TgramReplier interface (Telegram-n chatID/int replyToID signature) → generic inbound.Replier with SendReply(ctx, recipient commons.Recipient, body string, replyToID string, attachments []commons.Attachment) (s error) — recipient + string ids let any channel satisfy it. Config.TgramRep Config.Reply; NewDispatcher nil-check + Dispatcher.reply field Handle "reply" branch + both fastPath branches all migrated to build a commons.Recipient{Channel, ChannelUserID} + strconv.Itoa(replyToID) (" " when 0). pherald/cmd/pherald/li listenConfig.Subscriber (singular) → Subscribers map[string]func(ctx, InboundHandler) error; listenConfig inbound.TgramReplier → inbound.Replier; new loadEnabledCh parses HERALD_CHANNELS (comma-split, trim, dedupe; unset/empty → ["tgr Wave-6 single-channel behaviour preserved); new perChannelConfig(name namespaced env (HERALD_TGRAM_BOT_TOKEN/HERALD_TGRAM_CHAT_ID HERALD_SLACK_BOT_TOKEN/_APP_TOKEN/_CHANNEL_ID) → channel (fail-loud on missing required creds); loadListenConfigFromEnv resolves enabled channel via the channels.New(name, cfg) registry (T2), builds Subscribers[name]=ch.Subscribe + a channelReplier{ch} shim channels.Channel.SendReplyGeneric → inbound.Replier.Send the two intentionally-differently-named generic methods bridged) per channel, and

ID

Type Criticality Title

HRD-113 task high

them in a `channelRouter` that dispatches each reply to the adapter for recipient. `Channel` (fail-loud no replier for `channel %q` on unknown; never silently dropped). `runListen` fans in one goroutine per subscriber (manual `sync.WaitGroup` + buffered error channel; no new dep — `errgroup`-equivalent all feeding ONE dispatcher; the first non-cancel subscriber error cancels siblings (T11 chaos fail-loud). `tgram` is blank-imported in `listen.go` for its `init()` registration. `journal.go` `journalingReplier` migrated to wrap `inbound.Replier` (records `channel+chat_id+reply_to_message_` strings; JSONL kind kept `tgram.send_reply` for transcript back-compat). M resolution (Wave 7 plan Step 4): `inbound.Replier.SendReply` keeps the `SendReply` name (the `inbound` package's own, independent of `channels.Channel.SendReplyGeneric`); production wiring bridges via `channelReplier` shim so the journal decorator + dispatcher both stay `channel`. §107 evidence: new `TestRunListenMultiChannelFanIn` (2 stub subscribers `tgram+slack` each publish 1 event) asserts BOTH messages dispatched AND both recorded (`seen["ack (fake)"] >= 2`) — NOT "both goroutines started"; proven load-bearing (dropping the fan-in loop → `seen == map[]` → FAIL). E single-channel `TestListenWiresHandlerAndExitsOnCancel` + `TestRunListenRejectsBadConfig` migrated to the one-entry `Subscriber` still PASS. `recordingReplier/stubReplierJournal` stubs (`dispatcher_listen_test/journal_test`) migrated to the generic signature; numeric `chat_id/reply_a` preserved via `decode.go` `build ./pherald/... + go test -race -count=1 ./pherald/... ./commons_messaging/... all green; go clean; §107.y quiescence clean.`

Wave 7 T4 — generalize the bot self-filter via `BotSelfIdentity` (`channel-agnostic` file `commons_messaging/channels/selffilter.go` adds the `Raw-map` constants (`RawSenderIsBot/RawSenderIdentityKind/RawSenderIdentityStampSender` (`raw map[string]any, isBot bool, kind IdentityKind, value string`) (records the sender's native identity into `ev.Raw`; `nil-map` not `IsSelfEcho` (`ev commons.InboundEvent, self SelfIdentity`)). §32.9 anti-echo-loop guarantee made `channel-agnostic` — compares the right native `IdentityKind`: `username` for Telegram, `user_id` for Slack, `address` for Scope is deliberately narrow: a DIFFERENT bot is KEPT (multi-bot collaboration subscriber traffic — the §107 anchor that the filter is not over-broad); a human is empty `self.Value` never echoes (conservative KEEP → surfaces as duplicate silent loss; the real guard is `Subscribe` refusing to boot on empty `self`). `commons_messaging/channels/tgram/subscribe.go` migrated: `cap self := channels.SelfIdentity{Kind: channels.IdentityUser, Value: bot.Me.Username}` once at `Subscribe` boot (keeping the existing `ev` `username` boot-refusal echo-loop guard) and routes ALL FOUR live handlers (`OnPhoto/OnDocument/OnVoice`) through a new `stampAndIsSelfEcho` (`self`) helper that stamps the Telegram sender's `IsBot+@username` into `ev`. delegates to `channels.IsSelfEcho` — replacing the four Wave-6 `shouldDropBotSelf(msg, selfUsername)` early-returns. The Wave-6 `shouldDropBotSelf` func + its `TestSubscribeBotSelfFilter` + `SelfFilterForTest` (`export_test.go`) are PRESERVED BYTE-IDENTICAL M1 paired-mutation gate (`tests/test_wave6_mutation_meta.sh`) anchored exact-text `perl -i -0pe` regex on `shouldDropBotSelf`'s body, so keeping identical keeps M1 load-bearing (verified: the M1 regex still matches + applies its marker cleanly post-migration). The live path and the gate now express the SAME invariant — production on the generalized `IsSelfEcho`, the gate on the `tgram`. §107 evidence: `TestIsSelfEchoUsername` proves all three username cases

ID	Type	Criticality	Title
			DROPPED, different-bot KEPT, human KEPT; TestIsSelfEchoUserID proves Slack user_id kind (matching bot id DROPPED, different id KEPT); TestIsSelfEchoEmptySelfNeverEchoes proves empty-self KEEP; the helper builds events via the production StampSender (DRY — the test can never break the wire-key contract); TestSubscribeBotSelfFilter still PASSES (Wave 7 T9 e2e invariant preserved); go test -race -count=1 ./commons_messaging/... ./pherald/... all green; go vet clean. Live runtime evidence (real Telegram identity + own-message drop in the closed loop) lands via Wave 7 T9 e2e invariant E83. Wave 7 T3 — per-channel inbox subdirs ~/ .herald/inbox/<channel>/<sha256>.<ext>. New file commons_messaging/channels/inbox.<channel>.channel-agnostic InboxDir(channel string) (string, error) (returns ~/ .herald/inbox/<channel>/, MkdirAll 0700, empty-channel error) + WriteContentAddressed(channel, mime string, r io.Reader) (path, sha256Hex string, err error) (streams r into <dir>/<sha256>.<ext> via io.MultiWriter(tmp, sha256.New()) — never writes the full payload — then atomic temp+rename; idempotent: if the content-addressed file already exists the .part temp is dropped and the existing path returned unchanged) + MimeToExt(mime string) (the Wave-6 tgram-private mimeToExt switch was moved to VERBATIM to package level so every adapter shares one map). commons_messaging/channels/tgram/attachments.go DownloadAttachment now does Telegram-specific bot.File(&telebot.File{FileID}) fetch then delegates to stream-hash-rename to channels.WriteContentAddressed(string(common.ChannelToken, mime, rc) — so tgram attachments land under ~/ .herald/inbox/tgram/<channel> channel subdir, was the flat ~/ .herald/inbox/ in Wave 6). The tgram-private mimeToExt + the crypto/sha256/encoding/hex/io/os/path/filepath helpers were removed from attachments.go (the helper owns them now); the method (*Adapter).DownloadAttachment(T1) delegates to the package func unmarshal both forms route through the shared helper. §107 evidence: TestInboxDirIsPerChannel proves InboxDir("tgram") ends in . .tgram AND InboxDir("tgram") != InboxDir("slack") (no flat-inbox regression); TestWriteContentAddressedHashesAndIsIdempotent proves three §107 bluff classes on real on-disk state — (a) path shape under inbox/slack/<sha>.txt, (b) re-read on-disk bytes byte-equal the payload (sha in path == sha of content), (c) 2nd write of same content returns same (path, sum) and leaves zero residue (real os.ReadDir scan); the existing Wave-6 TestDownloadAttachmentContentAddressed (updated to the /tgram segment) still PASSES end-to-end against a httptest Bot API, proving the round-trip is behaviour-preserving. go test -race -count=1 ./commons_messaging/... ./pherald/... ./qaherald/... all green (no assumption regressions); go vet clean. Live runtime evidence (real Telegram attachments landing under inbox/tgram/) lands via Wave 7 T9 e2e invariant E83. Wave 7 T2 — channel registry (name→constructor) + init()-based registration commons_messaging/channels/registry.go adds Register(name Constructor) / New(name, Config) (Channel, error) / Names() ([]string, the ErrUnknownChannel sentinel, the channel-agnostic Config (Channel/Token/AppToken/Target/BaseURL/Extra), and the Constructor func(Config) (Channel, error) type — a sync.RWMutex-guarded map[string]Constructor. Register panics on duplicate (a typo in two init() would silently shadow one — a §107 bluff class this framework forbids in the qaherald scenario. Register pattern). New wraps ErrUnknownChannel (fmt.Errorf("%w: ...")) for unknown names — callers (pherald list
HRD-112	task	middle	
HRD-111	task	high	

ID

Type Criticality Title

HRD-110 task high

MUST surface it; a silent no-op channel is a §107 PASS-bluff. `commons_messaging/channels/tgram/tgram.go` gains an `init()` that calls `channels.Register(string(common.ChannelTelegram), ...)`. The constructor builds `NewAdapterWithBaseURL(cfg.Token, cfg.TargetURL, cfg.BaseURL)` when `cfg.BaseURL != ""` (http test seam) else `NewWithCreds(cfg.Token, cfg.Target)` — so a blank import self-reg adapter. §107 evidence: `TestRegistryResolvesRegisteredChannel` proves actually constructs + returns a working `Channel(c.Name()=="fake-rt", TestRegistryUnknownChannelErrors proves errors.Is(err, ErrUnknownChannel)` for unknown names; `TestRegistryDuplicatePanic` proves the duplicate-Register panic; `TestTgramRegisteredViaInit` (k _ ".../tgram") proves the `init()` actually registered tgram in `Names()` op — verified failing before the `init()` landed: `Names()=[dup-rt fake-tgram, PASS after)`. The shared `fakeChannel` test stub uses `SendReplyGeneric` T1-resolved interface method name, NOT `SendReply`). `go test -race ./commons_messaging/... green` (tgram regression-free); all 16 workspace modules build; `go vet ./commons_messaging/... clean`. Live runtime evidence (channel resolved by name + round-trip) lands via Wave 7 T9 e2e invariants E81/E82. Wave 7 T1 — extract `commons_messaging/channels.Channel` interface (refactor). New package `commons_messaging/channels/` defines `Channel` the §11.0 `commons.Channel` — Name/Capabilities/Send/Subscribe/HealthCheck the three inbound-runtime methods Telegram implicitly defined: `SendReplyGeneric` string-typed reply, `BotSelfIdentity` returning `SelfIdentity{Kind, Value, DownloadAttachment}`, the `SelfIdentity` value type, and the `Identity` constants (`IdentityUsername/IdentityUserID/IdentityAddress`). `*tgram.Adapter` gains `BotSelfIdentity` (`getMe` via `ensureBot`; empty-username §107 echo-loop-hazard error), `SendReplyGeneric` (string→int64/int adapter native Wave-6 `SendReply`), and a method-form `DownloadAttachment` (round-trip package-level func through `ensureBot`'s live bot) — so `var _ channels.Channel (*tgram.Adapter)(nil)` compiles. Naming divergence from plan Step 4 to + documented in `channel.go` package doc: the interface reply method is named `SendReplyGeneric` (not `SendReply`) so tgram satisfies it WITHOUT touching native, §107-mutation-gate-anchored int64/int `SendReply` (whose migration to signature is T5's scope) — keeping T1 a pure refactor with zero cross-package change evidence: `var _ channels.Channel = (*tgram.Adapter)(nil)` compiles. `TestTgramSatisfiesChannel + TestSelfIdentityType` PASS; full `-race ./commons_messaging/... green` (tgram regression-free — pure-proof); all 16 workspace modules build; `go vet ./commons_messaging/... clean`. Live runtime evidence (real `getMe` self-identity + reply round-trip) lands via Wave 7 T9 e2e invariant E81.

HRD-101 task high

Wave 6.5 ticket-lifecycle layer — §32.6 deterministic classifier (`Bug:/Issue:/Implementation:/Investigation:/Query:/Question:/Help:/Status:/Continue:/Done:/Resolve:/Reopen:/Override:` prefix routing with Type+Criticality+Confidence scores; natural-language no-prefix → query fallback) + `DocsIssueOpener` (§32.9 — real docs/Issues.md HRD-NNN allow concurrent-safe via lockfile) + `RunnerEventEmitter` (§33.7 — direct runner.RunCloudEvent dispatch, no HTTP loopback) + `Help:/Status:/Continue:` fast handlers (bypass Claude Code, sub-second latency — S2 proved 0.74s, no CC bootstrap overhead) + `Done:/Reopen:` atomic Issues[?] Fixed migration (rollback-via-sha) tgram. `SendReply` outbound attachment fan-out (multipart dispatch) + wiring into pherald `listen`. §107.x live evidence (docs/qa/HRD-101-

ID

Type Criticality Title

HRD-100 task high

lifecycle-2026-05-23T03-16-17-w6.5live/): S1 natural-language real Opus CC dispatch ~31s incl. bootstrap, threaded reply (wire-byte proof inbound message_id=30 → reply sent_message_id=31 with reply_to_message_id=30); S2 Help: fast-path — classifier Type: help Confidence: 1, CC dispatch ABSENT (event absence + 0.74s latency prove no BuiltinHelp catalogue text returned (reply_to_message_id=32 → sent_message_id=33); transcript.jsonl captures both directions of all four messages, query, Help:, Status: also exercised live). Reproducible test harness tests/test_wave6.5_lifecycle.sh covers 15 scenarios (Bug:/Task:/Query: + a Done:/Reopen: + classifier edge cases). Spec V3 r13: §32.6/§32.9/§33.6/§33.7 successfully landed. Tag v0.5.0.

Wave 6 — pherald inbound runtime + Claude Code headless bridge (code-doc closure Cobra subcommand pherald listen runs the closed-loop runtime: Telegram getUpdates long-poll (OnText/OnPhoto/OnDocument/OnVoice) + bot self-filter (bot.Me.Username anti-echo guard captured at Subscribe construction; refusal getMe returned no username) + sha256 content-addressed attachment download ~/.herald/inbox/<sha256>.<ext> (idempotent — duplicate download allowed) + write) + Claude Code dispatch with Opus pinned in the spawned argv (claude claude-opus-4-7 --resume <UUID> --print <envelope>) + envelope pre-text — verbatim operator wording ("We have received new message through communication channel <channel-name>. <classification> <sentence>. <attachment list>") preceding the <<<HERALD-DISPATCH v1>>> structured block + <<<HERALD-REPLY>>> parser with action routing default / issue.open / event.emit) + tgram.Adapter.SendReply(chatID, text, replyToID, attachments) wiring reply_to_message_id from the original message_id. --qa-out-dir <path> flag journals every outbound event as JSONL per §107.x. Closing 12 atomic Wave-6 commits T1=ac (HERALD_PROJECT_NAME resolver + .env.example), T2=0d55274 (Opus message_id), T3=14626f7 (envelope pre-text), T4=b4018a3 (bot self-filter), T5=d9156f1 (OnDocument/OnVoice + sha256 download), T6=a22a054 (pherald listen subcommand), T7=a3b238a (pherald/internal/inbound/Dispatcher routing), T8=5f92f8a (tgram.SendReply with reply_to_message_id), T9=f7913b8 (live closed-loop e2e script tests/test_wave6_live_loop.sh), T10a=520e0c6 (--qa-out-dir flag + JSONL journaling middleware), T11= (e2e_bluff_hunt E63-E70 invariants), T12=96c7c6b (Wave 6 mutation gate tests/test_wave6_mutation_meta.sh — 3 paired hermetic mutations). §107 event new e2e invariants E63-E70 land in this commit (all currently SKIP with documentation — E63 lifecycle / E64 bot self-filter / E65 attachment sha256 / E66 envelope pre-Opus argv / E68 reply_to_message_id wire-bytes / E69 action routing issue live closed-loop). Conversion SKIP → PASS lands at T10b when the operator exports HERALD_TGRAM_BOT_TOKEN + HERALD_TGRAM_CHAT_ID + HERALD_CL and runs bash tests/test_wave6_live_loop.sh --qa-out-dir <path> HRD-100-<run-id>/. Tag v0.4.0 is DEFERRED to T13b (post-T10b live closure §107.x) — code-doc closure ships here.

HRD-016 task middle

pherald /v1/events Runner wiring (REST API surface via Gin Gonic per spec §32 7-stage event-ingest pipeline lands as 7 concrete structs under pherald/internal/runner/ (EventParser, IdempotencyChecker, TenantResolver, PolicyGate, SubscriberResolver, ChannelDispatcher, OutcomeRecorder) orchestrated by runner.Runner.Run. pherald/internal/http/events.go exposes HTTP handler; pherald/cmd/pherald/{main,stubs}.go lazily builds JWT verifier inside the serve subcommand so other paths still work without HERALD_PG_DSN/HERALD_AUTH_MODE. HTTP contract: 202 Accepted + Reopen

ID	Type	Criticality	Title
HRD-011	task	middle	on fresh event, 200 + X-Herald-Replay: true on replay, 401 on missing/in 400 on parse failure. §107 evidence: 23/23 runner package tests green (per-stage integration scenarios: happy/duplicate/deny); 6 new e2e invariants E37-E42 (PG skipped when absent — honest SKIP-with-reason per §11.4.3); mutation gate te test_wave3_mutation_meta.sh gains M2 (Duplicate=false unconditional breaks), M4 (skip events_processed.Insert → E38/E42 break). M3 SKIP-with-reas Wave 3c deny-evaluator e2e. Commits T1..T10: eb9b2f4, 40ad60f, 2fdb7b 4c4af24, 3988114, 5468897, 6ea29a4, be71db1, c7b89a4, (this comm Telegram channel adapter live integration — telebot.v3 vendored (submodules telebot); HealthCheck + Send + Subscribe + outbound_delivery_evidence per live-wired with §107 bluff guards. Closed atomically with operator-session live e 2026-05-22: real bot delivery message_id=5 from @atmosphere_worker_ 2057253161 (validated via getChat) using the extended wizard CLI's - - bot (env: HERALD_TGRAM_BOT_TOKEN) + - - chat-id (validated via getCh persist) + - - non-interactive flags. Token→getMe→getChat→sendMessage confirmed end-to-end in commit 140a2f1 session. commons_auth/ JWT verifier — HMAC + JWKS hybrid + Gin middleware + cla vendored Herald-internal per §11.4.74 catalogue-check no-match. Catalogue evid docs/catalogue-checks/HRD-099-commons-auth.md. Renumbered HRD-093 (Wave 3a executing-time anchor) in the post-Wave-3a doc-cleanup pas commits dbbea95 . . 0cc6fad keep the old HRD-093 in their commit message sherald /v1/safety_state live — process-local Aggregator + background m (15s tick) + Handler with JWT gate via commons_auth.GinMiddleware. Replaces 501-stub. §107 evidence: e2e E46 (no-auth → 401), E47 (valid HMAC → 200 + current_mem_percent>0 + last_destructive_op=null + uptime_seconds>=1 + oper Paired mutation M5 (zero-mem Aggregator) proves E47 catches the regression. E with-reason — destructive-op trigger awaits HRD-033 body. cherald /v1/compliance live — paginated + filter-aware constitution_ surface with JWT gate via commons_auth.GinMiddleware. Backed by ConstitutionStore.ListQuery extended with Since/Until/Offset (commit 6276437) evidence: e2e E43 (no-auth → 401), E44 (valid HMAC + empty tenant → 200 + page=1 + page_size=50). E45 SKIP-with-reason — cross-binary cherald-reads-pl deferred to Wave 3b (Runner not live yet). commons/cli/ shared CLI scaffold (Wave 2) — NewRootCmd + VersionCmd + S (with Middleware hook) + StubCmd + healthz/readyz/metrics handlers. Vendored internal per §11.4.74 catalogue-check no-match. As-built evidence: §44.M of spec docs/catalogue-checks/HRD-092-commons-cli.md. sherald flavor scaffold — System Herald binary serving :24793 with cli.ServeCm stubs (HRD-033/034/040/046/056) + /v1/safety_state 501 stub → HRD-098. cherald flavor scaffold — Constitution Herald binary serving :24792 with cli.Serv §43 stubs + /v1/compliance 501 stub → HRD-028. bherald flavor scaffold — Build Herald CLI-only binary with 3 §43 stubs (HRD-035/041/045). rherald flavor scaffold — Release Herald CLI-only binary with 3 §43 stubs (HRD-031/032/045). iherald + scherald flavor scaffolds — Incident Herald serving :24794 with /v1/we → HRD-024 + Scheduled-audit Herald CLI-only with 1 §43 stub (HRD-047). Pa iherald has zero §43 stubs. Claude Code dispatcher live integration — claude - - resume <UUID> - - p <envelope> exec + <<<HERALD-REPLY>>> JSON parse +
HRD-099	task	low	commons_auth/ JWT verifier — HMAC + JWKS hybrid + Gin middleware + cla vendored Herald-internal per §11.4.74 catalogue-check no-match. Catalogue evid docs/catalogue-checks/HRD-099-commons-auth.md. Renumbered HRD-093 (Wave 3a executing-time anchor) in the post-Wave-3a doc-cleanup pas commits dbbea95 . . 0cc6fad keep the old HRD-093 in their commit message
HRD-098	task	middle	sherald /v1/safety_state live — process-local Aggregator + background m (15s tick) + Handler with JWT gate via commons_auth.GinMiddleware. Replaces 501-stub. §107 evidence: e2e E46 (no-auth → 401), E47 (valid HMAC → 200 + current_mem_percent>0 + last_destructive_op=null + uptime_seconds>=1 + oper Paired mutation M5 (zero-mem Aggregator) proves E47 catches the regression. E with-reason — destructive-op trigger awaits HRD-033 body.
HRD-028	task	low	cherald /v1/compliance live — paginated + filter-aware constitution_ surface with JWT gate via commons_auth.GinMiddleware. Backed by ConstitutionStore.ListQuery extended with Since/Until/Offset (commit 6276437) evidence: e2e E43 (no-auth → 401), E44 (valid HMAC + empty tenant → 200 + page=1 + page_size=50). E45 SKIP-with-reason — cross-binary cherald-reads-pl deferred to Wave 3b (Runner not live yet).
HRD-092	task	middle	commons/cli/ shared CLI scaffold (Wave 2) — NewRootCmd + VersionCmd + S (with Middleware hook) + StubCmd + healthz/readyz/metrics handlers. Vendored internal per §11.4.74 catalogue-check no-match. As-built evidence: §44.M of spec docs/catalogue-checks/HRD-092-commons-cli.md.
HRD-093	task	middle	sherald flavor scaffold — System Herald binary serving :24793 with cli.ServeCm stubs (HRD-033/034/040/046/056) + /v1/safety_state 501 stub → HRD-098.
HRD-094	task	middle	cherald flavor scaffold — Constitution Herald binary serving :24792 with cli.Serv §43 stubs + /v1/compliance 501 stub → HRD-028.
HRD-095	task	low	bherald flavor scaffold — Build Herald CLI-only binary with 3 §43 stubs (HRD-035/041/045).
HRD-096	task	low	rherald flavor scaffold — Release Herald CLI-only binary with 3 §43 stubs (HRD-031/032/045).
HRD-097	task	low	iherald + scherald flavor scaffolds — Incident Herald serving :24794 with /v1/we → HRD-024 + Scheduled-audit Herald CLI-only with 1 §43 stub (HRD-047). Pa iherald has zero §43 stubs.
HRD-012	task	middle	Claude Code dispatcher live integration — claude - - resume <UUID> - - p <envelope> exec + <<<HERALD-REPLY>>> JSON parse +

ID	Type	Criticality	Title
			<code>claude_code_sessions</code> persistence per §33. §107 evidence: Plan 2 Task 6 <code>702b5a3</code> (live PASS 24.24s — real claude CLI round-trip, structured reply parse Outcome+Summary non-empty) + Task 7 commit <code>4718c0e</code> (live PASS 36.23s — <code>session_state</code> upsert under <code>HeraldSystemTenant</code> with exact-equality assertions on <code>session_uuid</code> + <code>anchor_path</code> + <code>last_response</code> JSONB round-trip). HRD-085..HRD-086 open for upstream-defined <code>TaskRepository</code> methods not exercised by the Dispatch path. <code>commons_storage</code> live wiring — <code>pgx</code> pool + RLS-enforcing <code>WithTenantContext</code> (fixed RLS-bypass bug via E14 falsifiability) + 9 migrations + background queue (digital.vasic.background bound via <code>pgxTaskRepository</code>) + Redis ACL (digital.vasic.pherald migrate up/status/down/validate subcommand + 3 new §107 e2e invariants E16) + HRD-085..090 registered for queue-repository stubs Refine I6 inheritance-gate invariant from blanket <code>.gitmodules-forbidden</code> to "no <code>constitution/</code> entry in <code>.gitmodules</code>" — paired meta-test <code>test_i6_refinement_meta.sh</code> with 3 anti-bluff subtests. Enables Foundation Helix-stack submodule installs. Propagate Universal §11.4.73 (main-spec versioning + revision discipline) and §11.4.74 (submodule-catalogue-first discovery) into parent constitution <code>Constitution.md</code> + <code>CLAUDE.md</code> + <code>AGENTS.md</code> pherald CLI scaffold — Cobra root + version + 5 stubbed subcommands; <code>pherald version --json</code> returns canonical build info <code>commons_messaging</code> + <code>null://</code> adapter — full §11.0 Channel contract impl with <code>fail_rate/latency/ceiling</code> URL params + 8-case unit test suite <code>commons_prefix</code> module — §8.2 deterministic 3-letter prefix algorithm with Canonical + collision-resolution via <code>fnv1a32</code> + table-driven tests <code>commons</code> module — full §11.0 Go type contract (<code>Channel</code> + <code>Capabilities</code> + <code>Delivery</code> + <code>OutboundMessage</code> + <code>Body</code> + <code>Attachment</code> + <code>Recipient</code> + <code>Action</code> + <code>Priority</code> + <code>Recipients</code> + <code>InboundHandler</code> + <code>InboundEvent</code> + <code>Subscriber</code> + <code>SubscriberAlias</code> + <code>CloudEventEnvelope</code> + <code>TraceContext</code> + <code>Branding</code> + <code>ChannelID</code> + <code>PreferenceSet</code> + ...) + Clock abstraction helper + <code>DefaultBranding</code> factory V3 r3 cross-doc sync + tracking-doc scaffold (<code>Issues/Fixed/Status/Status_Summary/Issues_Summary/Fixed_Summary/CONTINUATION</code>) V3 r2 flavor refinement — 9 flavors × per-channel interaction tables V3 r1 operator-product layer (§31..§36 + §18.2 expansion) V2 r3 — Go type contract closure + operational ops detail V2 r2 — close prose^[2] definition gaps + add operational guidance V2 r1 — architectural authoring (<code>CloudEvents/Watermill/OTel/RLS/SLSA/9 flavors</code>) V1 — initial MVP specification + Review section + Recommendations
HRD-010	task	middle	
HRD-080	task	low	
HRD-017	task	middle	
HRD-014	task	middle	
HRD-013	task	middle	
HRD-009b	task	low	
HRD-009	task	middle	
HRD-007	task	middle	
HRD-006	task	middle	
HRD-005	task	middle	
HRD-004	task	middle	
HRD-003	task	middle	
HRD-002	task	middle	
HRD-001	task	middle	